

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA0304 COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

***CO1:** Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 40%

Dr. Dumpala Prasanth

QUESTIONS: 10

Total Marks: 50

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action selection methods.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Code of Conduct:

I K Varshan (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Answers

Experiment 1: Installation of RL Development Environment

Aim: Install Anaconda, Python, TensorFlow, Keras, Gymnasium, NumPy, Matplotlib and verify with a simple RL program.

```
python import gymnasium
```

```
as gym import numpy as
```

```
np import tensorflow as tf
```

```
print("TensorFlow:", tf.__version__)
```

```
print("Gymnasium:", gym.__version__) print("NumPy:",  
np.__version__)
```

```
env = gym.make("CartPole-v1")
```

```
state, info = env.reset()
```

```
total_reward = 0 for _
```

```
in range(20):
```

```
    action = env.action_space.sample()    state, reward, terminated,
```

```
truncated, info = env.step(action)    total_reward += reward    if
```

```
terminated or truncated:        state, info = env.reset()
```

```
print("Environment working. Total reward collected:", total_reward)
```

```
env.close()
```

Expected Output: Library versions printed; environment runs without error; a total reward value is displayed.

Experiment 2: Familiarization with Gymnasium Environments

Aim: Create, initialize, and interact with FrozenLake, CartPole, and MountainCar.

```
python import gymnasium as gym

def explore_env(name, episodes=1, steps=10):

    env = gym.make(name)    print(f"\n---
{name} ---")    print("State space:",
env.observation_space)    print("Action
space:", env.action_space)    for ep in
range(episodes):        state, info = env.reset()
print("Initial state:", state)        for t in
range(steps):

        action = env.action_space.sample()        next_state, reward,
terminated, truncated, info = env.step(action)

        print(f'Step    {t}:    action={action},    next_state={next_state},
reward={reward}, done={terminated or truncated}')

        if terminated or truncated:

            break

    env.close()

explore_env("FrozenLake-
v1")

explore_env("CartPole-
v1")
```

```
explore_env("MountainCa  
r-v0")
```

Expected Output: For each environment — state/action space description, and a step-by-step trace of states, actions, rewards.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: Implement a basic agent–environment interaction loop with states, actions, rewards, and a policy. python import random

```
class SimpleEnvironment:
```

```
    def __init__(self):    self.states = [0,  
1, 2, 3, 4] # 4 = goal    self.state = 0
```

```
    def reset(self):  
self.state = 0    return  
self.state
```

```
    def step(self, action):  
        # action: 0 = stay, 1 = move forward  
if action == 1 and self.state < 4:  
self.state += 1    reward = 10 if self.state  
== 4 else -1    done = self.state == 4  
return self.state, reward, done
```

```

class Agent:
    def
    policy(self, state):

        return 1 # always move forward

def run_episode():
    env =
    SimpleEnvironment()
    agent
    = Agent()
    state = env.reset()
    total_reward = 0
    print("Start
    state:", state)
    while True:

        action = agent.policy(state)
        state, reward, done =
        env.step(action)
        total_reward += reward

        print(f"Action={action} -> State={state}, Reward={reward}")

        if done:

            break

    print("Episode finished. Total reward:", total_reward)
    run_episode()

```

Expected Output: Sequence of state transitions from 0 to goal state 4, with reward printed each step, ending with total reward.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: Model state transitions, reward functions, and policy execution for an MDP.

```
python import random
```

```

states = ["S0", "S1", "S2", "Goal"] actions
= ["A", "B"]

# transition probabilities: P[state][action] = [(prob, next_state, reward)]

P = {
    "S0": {"A": [(1.0, "S1", -1)], "B": [(1.0, "S0", -1)]},
    "S1": {"A": [(0.8, "S2", -1), (0.2, "S0", -1)], "B": [(1.0, "S1", -1)]},
    "S2": {"A": [(1.0, "Goal", 10)], "B": [(1.0, "S1", -1)]},
    "Goal": {}
}

policy = {"S0": "A", "S1": "A", "S2": "A"}

def simulate(max_steps=10):
    state = "S0"    total_reward = 0    for step in range(max_steps):
        if state == "Goal":            break        action = policy[state]
        outcomes = P[state][action]    probs = [o[0] for o in outcomes]
        idx = random.choices(range(len(outcomes)), weights=probs)[0]
        _, next_state, reward = outcomes[idx]    print(f"Step {step}: {state} --
{action}--> {next_state}, reward={reward}")    total_reward += reward
        state = next_state    print("Total reward:", total_reward)

simulate()

```

Expected Output: A trace of state transitions following the given policy and transition probabilities, ending at "Goal", with cumulative reward.

Experiment 5: Bellman Equation Demonstration

Aim: Implement Bellman Expectation and Bellman Optimality equations; observe convergence of value estimation. python import numpy as np

```
states = [0, 1, 2, 3] # 3 = terminal
```

```
actions = [0, 1] gamma = 0.9
```

```
# transition[state][action] = (next_state, reward) transition
```

```
= {
```

```
    0: {0: (0, -1), 1: (1, -1)},
```

```
    1: {0: (0, -1), 1: (2, -1)},
```

```
    2: {0: (1, -1), 1: (3, 10)},
```

```
    3: {0: (3, 0), 1: (3, 0)}
```

```
}
```

```
V = np.zeros(len(states))
```

```
# Bellman Expectation Equation (uniform random policy)
```

```
print("---- Bellman Expectation Equation ----") for i in
```

```
range(50):    new_V = np.copy(V)
```

```
    for s in states[:-1]:
```

```

        value = 0

    for a in actions:

        ns, r = transition[s][a]        value

    += 0.5 * (r + gamma * V[ns])

    new_V[s] = value    V = new_V

    print("State values (expectation):", V)

# Bellman Optimality Equation V_star =
np.zeros(len(states)) print("\n---- Bellman
Optimality Equation ----") for i in range(50):

    new_V = np.copy(V_star)

    for s in states[:-1]:

        q_values = []        for

        a in actions:

            ns, r = transition[s][a]

        q_values.append(r + gamma * V_star[ns])

    new_V[s] = max(q_values)    V_star = new_V

    print("Optimal state values:", V_star)

```

Expected Output: Two arrays of converged state values — one under a random policy (Expectation), one optimal (Optimality); optimal values are $\geq$ expectation values.